

Workflow Failure Modes I: Bad Inputs and False Confidence

FRIDAY AI Education — Episode 029 · 2026-07-15 · Printable Companion

FRIDAY AI Education — Episode 029 · Companion Paper

AI-Native Operator arc, lesson 15 of 36 · Unit: Workflow Failure Modes, part 1 of 2

Abstract

An AI workflow can fail before it ever takes an action. It can reason from context that used to be true, from a source that was never right, or from no source at all — and then present the result with the same fluent confidence in every case. This paper builds a working taxonomy of these *pre-action* failures: **staleness** (true then, false now), **source failure** (the input was wrong or wrongly chosen), and **hallucination** (the output was never grounded in anything). It then examines the property that makes all three dangerous rather than merely annoying: **calibration**, the relationship between how confident a system sounds and how often it is right. The central claim is that these four failure modes produce nearly identical-looking outputs, so diagnosing them requires tracing provenance rather than judging prose. The paper closes with three worked examples from ordinary company operations — a bad brief, a wrong lead, an unsupported recommendation — and a small set of design rules for workflows that fail loudly instead of confidently. Part II of this unit will move downstream: what happens when a bad output becomes an unsafe action, and how systems drift and recover.

1. Where This Sits

The last three lessons were about operating memory: what deserves to be remembered, who has authority over a fact when sources conflict, and how retrieval and forgetting form a learning loop. The through-line of that unit was **provenance** — every fact in a serious system should carry a record of where it came from and when.

This lesson is the payoff of that discipline. Provenance is what makes failure *diagnosable*. When a workflow produces a wrong answer, the only reliable way to find out what went wrong is to walk the chain backward: output → reasoning → retrieved context → source → world. A system without provenance gives you a wrong answer and no thread to pull. A system with provenance gives you a wrong answer and a crime scene.

Today's new layer is the taxonomy of what you find when you pull that thread. The next lesson (Workflow Failure Modes II) covers what happens after: outputs that trigger unsafe actions, workflows that drift over time, and how recovery works. Today we stay upstream of action, where the damage is still only informational — which is exactly where you want to catch it.

One framing note before we start. None of these failure modes is exotic. Human organizations exhibit all four constantly: employees work from outdated org charts, cite the wrong memo, misremember meetings, and present guesses as facts in a confident tone. What changes with AI workflows is *rate and uniformity*. A language model produces confident prose at industrial scale, in the same polished register whether it is right or wrong. The failure modes are old. The throughput is new.

2. The Shape of the Problem: Failure Upstream of Action

Consider a minimal AI workflow, the kind you would build for any real operational task:

1. **Gather** — retrieve context: documents, records, memory, live data.
2. **Reason** — a model transforms that context into an output: a brief, a ranking, a recommendation.
3. **Present** — the output goes to a human or to a downstream step.

Every failure in today's lesson lives in this pipeline, and each of the three core failure modes attacks a different stage:

- **Staleness** is a failure of the *gather* stage relative to time. The context was accurate when written; the world moved.
- **Source failure** is a failure of the *gather* stage relative to selection. The system pulled the wrong document, the wrong record, the wrong person — or a source that was simply incorrect from the start.
- **Hallucination** is a failure of the *reason* stage. The model produced content that no input supports. The gather stage may have been flawless; the model filled a gap with fluent invention anyway.

And then there is the fourth concept, which is not a stage failure but a property of the whole system:

- **Calibration** describes whether expressed confidence tracks actual correctness. A poorly calibrated system doesn't just fail — it fails *while sounding exactly like it does when it succeeds*.

Hold onto that last sentence, because it is the operational heart of this lesson. If bad outputs looked bad, none of this would matter much. You would glance at them, wince, and ask again. The problem is that a brief built on a two-year-old company profile, a brief built on the wrong company's profile, and a brief built partly on invented facts are all — to a reader — the same artifact: clean paragraphs, specific claims, a confident register. **The failure modes are distinguishable only by their provenance, not by their surface.** That is why the diagnostic skill this lesson teaches is not "learn to spot bad text." Nobody can do that reliably, including the people who claim they can. The skill is: *know the four ways an output goes wrong, and know which question exposes each one.*

The four questions, which the rest of this paper unpacks:

1. **When** was this context true? (staleness)
2. **Which** source did this come from, and was it the right one? (source failure)
3. **Where** in any source does this specific claim appear? (hallucination)
4. **How often** is this system right when it sounds this sure? (calibration)

3. Staleness: True Then, False Now

Staleness is the most forgivable failure mode and, for that reason, the most persistent. Nothing malfunctioned. Nobody wrote anything false. A fact was recorded accurately, stored faithfully, retrieved correctly — and the world changed underneath it.

The mechanism is simple: **facts have shelf lives, but most systems store them without expiry dates.** A memory substrate that records "Company X has 40 employees" is recording a snapshot, not a truth. Retrieved eighteen months later, it is presented with exactly the same authority it had on day one. The retrieval step has no native concept of decay; a highly relevant stale fact will beat a moderately relevant fresh one every time, because relevance

is what retrieval optimizes for. Semantic similarity does not measure age.

Two properties make staleness insidious:

Staleness is invisible at the point of use. A stale fact is not malformed. It reads exactly like a fresh fact, because it was a fresh fact. There is no textual signal to catch. The only defenses are metadata (when was this recorded?) and domain judgment (how fast does this kind of fact change?).

Facts decay at wildly different rates. A company's founding year is effectively permanent. Its headcount decays over quarters. Its open roles decay over weeks. A person's title decays whenever they get promoted, fired, or bored. A competent operator carries an intuitive half-life for every category of fact; most AI workflows carry none. This is a design gap, not a model limitation — nothing prevents a system from tagging facts with recorded-at timestamps and expected volatility. Most systems just don't, because it is unglamorous work, and unglamorous work is where reliability actually lives.

The company-building translation: any workflow that touches **fast-moving external reality** — competitors, prospects, prices, people, regulation — is structurally exposed to staleness, and the exposure grows with every month the underlying knowledge base is not refreshed. A knowledge base is not an asset that appreciates by sitting still. Unattended, it quietly converts from an asset into a liability with excellent search.

The prior unit's authority-and-conflict material connects here in one sentence: recency is one of the standard tie-breakers when memory holds conflicting versions of a fact, and staleness is what happens when there is no conflict to break — the old version is the *only* version, so it wins by default.

4. Source Failure: The Right Answer to the Wrong Question

Source failure is broader than staleness, and it splits into two distinct sub-cases worth keeping separate.

Case one: the source is wrong. The document contains an error. The scraped page was a parody, a draft, or an SEO farm. The transcript was auto-generated and mangled the key number. The system did everything right and inherited a defect from upstream. This is the classic garbage-in problem, and its defense is source *quality* discipline: knowing which origins you trust, at what level, for what kinds of claims. This is precisely the authority-and-provenance machinery from the memory unit, applied at ingestion time.

Case two: the source is right, but it's the wrong source. This one is subtler and, in practice, more common in retrieval-backed workflows. Every fact in the retrieved document is true — the document is just about the wrong thing. The wrong John Smith. The parent company instead of the subsidiary. The 2024 pricing page instead of the current one. The European entity's terms instead of the American one's.

Case two is dangerous for a specific structural reason: **it defeats fact-checking.** If you take a suspicious output and verify its claims, they check out — because they're true. True about something else. The error lives at the *selection* step, one layer above where verification naturally looks. An operator who diligently confirms three facts from a wrong-source brief comes away *more* confident in a wrong document. Verification without checking identity is worse than no verification, because it launders the error.

The mechanism behind wrong-source retrieval is worth one paragraph, because it explains why this failure is systematic rather than freak. Retrieval systems match on similarity, and entities in the same industry, with similar names, or in adjacent roles are — by construction — similar. The features that make two companies confusable to a retrieval system are the same features that make them confusable to a distracted human. The difference is that the human sometimes notices the itch of "wait, is this the same company?" and the pipeline, unless explicitly built to,

never does. Entity resolution — *is this document about the thing I asked about?* — is a distinct checking step, and pipelines that skip it are betting the workflow on nobody ever having a common name.

The diagnostic question for source failure is not "is this claim true?" but two sharper ones: **"is this source trustworthy?"** and **"is this source about the right entity?"** Both must pass. Either alone is insufficient.

5. Hallucination: Content From Nowhere

Hallucination is the failure mode with the most press coverage and the most muddled public understanding, so it is worth being precise.

A language model is a generation engine. Given context, it produces the most plausible continuation — where "plausible" is learned from vast quantities of text. This is the same capability that lets it write, summarize, and reason. **Hallucination is not a malfunction of this machinery; it is this machinery operating normally in the absence of grounding.** When the context contains the answer, plausible generation tends to track the context. When the context has a gap — a missing number, an unstated name, an unasked-for level of detail — the machinery does not stop at the gap's edge. It generates the most plausible filler: a revenue figure that sounds right for a company of that size, a citation formatted exactly like real citations, a co-founder who plausibly could exist. The model is not lying, because lying requires knowing the truth and departing from it. It is doing what it always does — producing plausible text — in a place where plausible and true have come apart.

Three practical consequences follow from this mechanism:

Hallucination concentrates at specificity gaps. Broad claims are usually well-supported by training or context; it is the precise, checkable details — numbers, dates, names, quotes, URLs — where invention concentrates, because those are exactly the details context is most likely to lack and plausibility is least likely to nail. This gives you a triage rule: in any AI-generated document, the specific checkable claims deserve the most suspicion, not the least. This inverts the human heuristic, where specificity usually *signals* knowledge. A person who quotes exact figures generally has a source. A model that quotes exact figures has, at minimum, a strong sense of what exact figures look like.

Hallucination is bounded by grounding, not by scolding. Instructing a model to "not make things up" helps at the margin; it does not change the mechanism. The reliable lever is architectural: constrain outputs to be assembled from supplied evidence, and make "the context does not say" an acceptable — rewarded, even — answer. A workflow that punishes "I don't know" is a workflow that has explicitly ordered the plausible filler.

Hallucination and source failure compound. Retrieve the wrong document, and the model now generates plausibly *within the wrong frame*, smoothing over inconsistencies between the wrong source and the question asked. The smoothing is the model working as designed. The result is an output that is wrong twice, seamlessly.

The diagnostic question: **"show me where in the source material this specific claim appears."** Not "does this seem right" — *point to it*. A claim that cannot be pointed to is unsupported, whatever its truth value happens to be. Some unsupported claims are true by luck. An operating company does not run on luck it cannot distinguish from process.

6. Calibration: The Failure That Weaponizes the Others

Now the fourth concept, which is different in kind. Staleness, source failure, and hallucination are about *whether the output is right*. Calibration is about *whether you can tell*.

A system is well calibrated when its expressed confidence matches its actual accuracy: of the claims it makes with apparent near-certainty, nearly all are correct; of the claims it hedges on, an appropriately smaller fraction are. Calibration is not the same as accuracy. A system that is right 70% of the time and says so is honest and usable — you know exactly how much verification to apply. A system that is right 70% of the time and sounds right 100% of the time is a trap, because it gives you no signal about where to spend your scarce checking attention.

Language models have a specific, structural calibration problem: **the confidence of the prose is a stylistic property, not an epistemic one**. Fluency, declarative sentences, precise formatting, the absence of hedging — these are features of the *register* the model writes in, learned from text that mostly presents conclusions rather than doubts. The polish of the output carries almost no information about the reliability of the content. A hallucinated statistic arrives in the same confident sentence structure as a well-sourced one, because sentence structure is not where the model's uncertainty lives.

This is the precise sense in which calibration is the amplifier of the other three failures. Consider the counterfactual: if stale facts arrived flagged as old, wrong-source material arrived flagged as uncertain matches, and ungrounded claims arrived hedged, each failure would be an inconvenience — the system would be *telling you where to look*. Miscalibration removes the flags. It converts three detectable failures into three silent ones, and it does so uniformly, which is why "just read the output carefully" is not a control. There is nothing in the output to read carefully *for*.

Two consequences for how you evaluate any AI system, vendor, or internal workflow:

Demand the error distribution, not the demo. A demo shows you accuracy on chosen examples. Calibration only reveals itself over a *population* of outputs: when this system expresses certainty, how often is it wrong? That number is measurable — it is one of the core purposes of running evaluations — and any workflow doing consequential work should have it measured. A team that cannot tell you their system's confident-error rate has not measured it, and a system whose confident-error rate nobody knows should be treated as miscalibrated by default.

Calibrate the humans too. The failure of record is rarely "the model said something false." It is "the model said something false, and a person repeated it in a decision-making context because it sounded authoritative." Miscalibrated systems induce miscalibrated operators. The countermeasure is cultural and cheap: inside a company, an AI-generated claim is a *draft claim* until a human or a verification step has traced it to a source. Not because the model is usually wrong — it usually isn't — but because "usually" is not a provenance.

7. The Taxonomy at a Glance

Failure mode	Stage attacked	Root cause	Diagnostic question	Primary defense
Staleness	Gather (time)	World changed; fact didn't	<i>When was this recorded, and what's this fact's half-life?</i>	Timestamps, freshness policy, scheduled refresh
Source failure — bad source	Gather (quality)	Origin was wrong from the start	<i>Do we trust this origin, for this kind of claim?</i>	Source authority tiers at ingestion
Source failure — wrong source	Gather (selection)	True document, wrong entity	<i>Is this actually about the thing I asked?</i>	Entity resolution / identity check before use
Hallucination	Reason	Generation filled a grounding gap	<i>Point to where the source says this.</i>	Evidence-constrained output; "not in context" allowed

Failure mode	Stage attacked	Root cause	Diagnostic question	Primary defense
Calibration (meta)	Whole system	Confidence is style, not epistemics	<i>When it sounds sure, how often is it right?</i>	Measured confident-error rate; treat outputs as drafts

The table is a diagnostic order of operations, roughly cheapest-first: check the timestamp, check the entity, check the grounding, and maintain the measured error rate in the background as your prior. In a real incident you rarely know which row you're in until you've walked the provenance chain — which is the point of the next section.

8. Three Worked Examples

Abstractions earn their keep in the field. Here are the three canonical incidents, each dominated by a different failure mode, each ending with the trace that identifies it.

8.1 The bad brief

A partner meeting is on the calendar. A workflow assembles a briefing document on the other company: what they do, who's who, recent developments, suggested talking points. The brief is excellent — well-structured, specific, confident. In the meeting, the operator congratulates the other side on a product line they discontinued last year and asks for an introduction to an executive who left in the spring. The meeting recovers, barely. The impression of "this person did their homework on someone else's company" does not.

The trace: every claim in the brief resolves to a real source — a knowledge base profile ingested fourteen months earlier and never refreshed. Nothing was invented; nothing was mis-selected. Every fact was true at write time. **Diagnosis: staleness**, pure. The fix is not a better model or a better prompt; the model performed flawlessly on the context it was given. The fix is a freshness contract: people-and-product facts about an active counterparty have a shelf life measured in weeks, and a brief for a live meeting should either verify against current sources or state, prominently, the age of what it's built on. A brief that opened with "profile last refreshed 14 months ago" would have triggered a five-minute check. The information to trigger that check existed in the system the entire time. It just wasn't surfaced — which makes this, like most staleness incidents, a design failure wearing a data failure's clothes.

8.2 The wrong lead

A sales-research workflow qualifies inbound interest. A promising signal arrives from someone at "Meridian" — and the workflow, matching on name and industry, retrieves and reasons over Meridian Systems, a 500-person enterprise, when the actual prospect is Meridian Labs, an eleven-person startup. The qualification memo is impressive: right-sized deal estimate, org-chart-informed stakeholder map, tailored pitch angles. Every fact in it is true. The outreach lands with the confused eleven-person startup as a pitch evidently written for someone else, and a genuinely good lead goes cold on the reasonable inference that this vendor doesn't pay attention.

The trace: the timestamps are fine; the sources are reputable; every claim points cleanly to real material. The break is one step higher: the *selection* step matched an entity on similarity and never confirmed identity. **Diagnosis: source failure, wrong-source variant** — and note what the trace demonstrates: this incident survives fact-checking. Three spot-checked claims from that memo all verify, because they're true about Meridian Systems. The only check that catches it is the identity check: *does the retrieved profile's domain, size, and location match the actual inbound signal?* One field comparison. The fix is an entity-resolution gate between retrieval and reasoning — mechanically trivial, and structurally the only place this class of error can be caught, because downstream of it, everything is internally consistent.

8.3 The unsupported recommendation

An analysis workflow evaluates whether to enter a market segment. The resulting memo recommends entry, citing the segment's growth rate, a competitor's churn problem, and a favorable regulatory change — with numbers. The memo circulates; the numbers get repeated in a board conversation. Then someone preparing the follow-up asks the question that should have been asked first: *where do these figures come from?*

The trace: the growth rate traces to a real report — from three years ago (staleness, as a supporting actor). The regulatory change traces to a real proposal — that was never adopted (source failure, minor role). And the competitor churn figure traces to *nothing*. No document in the context contains it. It is a plausible number, formatted like data, generated to complete an argumentative structure that needed a churn figure at that point in the paragraph.

Diagnosis: hallucination in the lead role — with miscalibration as the accomplice that let all three defects travel. The memo expressed no gradation of confidence anywhere: the well-sourced claims and the invented one sat in identical declarative sentences, so the reader had no way to allocate suspicion. The fix is an evidence contract on the *output* side: a recommendation memo must carry claim-level citations, and a claim without a source either gets flagged as an assumption or doesn't ship. Under that contract, this memo would have arrived with one visible hole in it — which is precisely the information the board conversation needed.

Notice the pattern across all three: **the failure was identified by tracing, not by reading.** Each document read impeccably. The diagnosis came from asking *when* (the brief), *which* (the lead), and *where exactly* (the recommendation). Those three questions, plus the standing fourth — *how often is this system wrong when it sounds like this?* — are the whole diagnostic kit.

9. Design Rules: Workflows That Fail Loudly

The lesson so far is diagnostic. The design corollary can be stated compactly, because it follows directly from the taxonomy. A workflow resistant to these failures has four properties:

1. **Facts carry timestamps and half-lives.** Every stored fact records when it was captured; every category of fact has an expected volatility; retrieval surfaces age alongside content, and consequential outputs disclose the age of what they're built on. (Defends against staleness.)
2. **Sources carry authority, and selections carry identity checks.** Ingestion tags where material came from and how much it's trusted; any retrieval keyed to an entity confirms the entity before reasoning begins. (Defends against both source-failure variants.)
3. **Outputs carry evidence, and "not in context" is a valid answer.** Claims link to the passages that support them; the workflow rewards a declared gap over a plausible fill. (Defends against hallucination.)
4. **Confidence is measured, not performed.** The system's confident-error rate is a known, tracked number from evaluations, and humans treat unverified output as draft-grade regardless of how it reads. (Defends against miscalibration — the amplifier.)

For the FRIDAY architecture specifically, this is why the memory unit came *before* this one: GBrain's provenance, authority, and freshness metadata are not record-keeping hygiene — they are the load-bearing inputs to every diagnostic question in this paper. An orchestrator can only ask "when was this true?" of a substrate that stored the answer. And for the public-content knowledge-base wedge, this taxonomy is close to a product spec: the difference between "searchable content" and "operationally useful, cited knowledge" is exactly rules one through three, made visible to the user. The wedge is credible precisely because most of what exists ships without them.

A closing note on scope, and an honest limitation. Everything above governs failures that end in a *document* — a brief, a memo, a ranking that a human still reads before anything happens. The controls in this lesson (provenance, evidence contracts, calibration measurement) all assume there is a moment between output and consequence where a question can be asked. The next lesson removes that assumption: when workflows take actions — send the email, update the record, move the money — a bad input stops being a bad paragraph and becomes a bad event, and a new set of controls (permissions, approval gates, reversibility, recovery) takes over. Today's material is the foundation, because an action-taking system with unsolved input failures is just this lesson's incidents with the safety margin deleted.

10. Vocabulary

Staleness — A fact that was accurate when recorded but no longer reflects the world. Not an error at write time; an error at use time. Governed by the fact's half-life and the system's refresh policy.

Fact half-life — The characteristic rate at which a category of fact decays: founding dates barely decay; headcounts decay over quarters; job titles and open roles decay over weeks. A design input, not a metaphor.

Source failure — Reasoning from defective input: either a source that was wrong from the start (*bad source*) or a correct source about the wrong entity (*wrong source*). The wrong-source variant survives fact-checking, because its claims are true — about something else.

Entity resolution — The explicit check that retrieved material is about the same real-world entity as the question. The only control that catches wrong-source failures, because everything downstream of selection is internally consistent.

Hallucination — Model-generated content unsupported by any provided or reliable source; the generation machinery filling a grounding gap with plausible material. Concentrates at specificity gaps — numbers, names, dates, citations.

Grounding — The property that an output's claims are traceable to supplied evidence. The architectural (not rhetorical) remedy for hallucination.

Calibration — The degree to which a system's expressed confidence matches its actual accuracy. A well-calibrated system that is often wrong is still usable; a miscalibrated one that is usually right is a trap.

Confident-error rate — Of the outputs a system delivers with apparent certainty, the fraction that are wrong. Measurable through evaluations; the single most decision-relevant reliability number for an operator.

Provenance (from the memory unit) — The recorded chain of where a fact came from and when. The precondition for diagnosing any of the above; without it, all four failure modes are indistinguishable.

Evidence contract — A workflow rule that consequential outputs must carry claim-level links to their sources, with unsourced claims flagged as assumptions or excluded.

11. Reading Guide

NIST AI Risk Management Framework — <https://www.nist.gov/itl/ai-risk-management-framework>

The AI RMF is the U.S. standards body's voluntary framework for managing AI risk, and it is the closest thing to a shared vocabulary between builders, buyers, and (increasingly) regulators. Do not read it cover to cover; it is a framework document and reads like one. Read it *for structure*, about 45 minutes:

- **Start with the four core functions — Govern, Map, Measure, Manage.** Notice how today's lesson maps onto them: this paper's taxonomy is *Map* work (knowing where failures come from), the confident-error rate is *Measure* work, and the design rules in section 9 are *Manage* work. Govern is the standing organizational layer that makes the other three someone's actual job.
- **Then find the trustworthiness characteristics** (the framework's list includes *valid and reliable*, *accountable and transparent*, and others). Read "valid and reliable" as the formal home of everything in this paper, and "accountable and transparent" as the formal home of provenance.
- **How to read it as a founder:** not as compliance homework, but as a preview of the questions serious customers and partners will ask about any AI system you sell them. A company that can answer "how do you know when it's wrong?" in NIST's vocabulary is a company that clears procurement conversations its competitors stall in. The framework's quiet thesis — risk is managed by process and measurement, not by model choice — is also this paper's thesis, arrived at independently, which is a decent sign it's true.

12. Walking Questions

1. **Recall:** State the four failure modes and, for each, the single diagnostic question that exposes it. Could you do this at a whiteboard, uninvited, in a meeting where someone just said "the AI got it wrong"?
2. **Discrimination:** The wrong-lead example *survives fact-checking* and the unsupported recommendation *fails it*. Explain why, mechanically. If that distinction is blurry, reread sections 4 and 5 — it is the sharpest edge in this lesson.
3. **Application:** Pick one workflow you actually run or plan to run. Which of the four modes is it *most* exposed to, and what is the cheapest single gate — a timestamp disclosure, an identity check, a citation requirement — that would catch it?
4. **Systems:** Where in GBrain's existing metadata (provenance, authority, recency) do the four diagnostic questions already have answers — and which question currently has nowhere to look?
5. **Test:** What is one bounded, low-stakes workflow where you could deliberately *measure* a confident-error rate this month, rather than estimating it?

One-Sentence Takeaway

Stale context, wrong sources, and hallucinated claims all produce the same confident-looking output, so reliability comes not from reading harder but from building workflows where every claim can be traced to a source, a timestamp, and a measured error rate.